BÖLÜM 6: GÜVENILIRLIK VE HATA TOLERANSI — ÇÖKÜŞ İÇIN TASARLAMAK 💣 9 dk • DERS 23

Parça 23: Kaos Mühendisliği (Chaos Engineering): Sistem Sizi Çökertmeden Önce Onu Bilerek Bozmak

Netflix'in Chaos Monkey aracı sektöre yıkılmaz mimariler kurmayı nasıl öğretti?

> 🔥 **SAHADAN YAŞANMIŞ KRİZ HİKAYESİ**
>
> 2011 yılında Netflix, geleneksel IT yöneticilerini dehşete düşüren bir açıklama yaptı: 'Chaos Monkey' adında bir yazılım yazmışlardı ve bu yazılım mesai saatleri içinde prodüksiyonda rastgele sunucuları çatır çatır öldürüyordu! Geleneksel yöneticiler buna 'çılgınlık' dedi. Ancak Netflix mimarlarının muhteşem bir vizyonu vardı: Mühendisler masalarında kahvelerini yudumlarken sunucuların sürekli öldüğü bir ortamda, hiç kimse tek bir sunucuya bağımlı kırılgan bir sistem yazamazdı! 2012 yılında tüm bir AWS veri merkezi çöktüğünde, rakipler günlerce kapalı kalırken Netflix tek bir kullanıcının bile filmini durdurmadan yayına devam etti.

Zihniyet Dönüşümü: Kod Yazarlığından Mimarlığa

Kaos Mühendisliği rastgele ortalığı kırıp dökmek değildir; bir sistemin prodüksiyondaki çalkantılı koşullara dayanma kabiliyetine güven inşa etmek için yapılan bilimsel bir deney disiplinidir. Bir hipotez kurarsınız ('Eğer Redis ana sunucusu çökerse, uygulama 200ms gecikmeyle kopyalardan okumaya devam eder'), kontrollü bir hasar yarıçapında arızayı enjekte edersiniz, telemetriyi izlersiniz ve zayıflığı gece saat 03:00'te bir kriz patlamadan önce mesai saatinde tespit edip düzeltirsiniz.

Temel Mimari İlkeler ve Zihinsel Modeller

Kaos Mühendisliğinin Bilimsel Metodu

1. **Durağan Durumu Tanımla (Steady State):** Normal durumdaki temel metrikleri ölç (Örn: Dakikadaki sipariş, P99 gecikme < 300ms, hata oranı < %0.1).
2. **Hipotez Kur:** Arıza anında durağan durumun korunacağını öngör.
3. **Kontrollü Hata Enjekte Et:** Gerçek dünya felaketini simüle et (bir veritabanı düğümünü kapat, ağa 500ms gecikme ekle).
4. **Durağan Durumu Doğrula:** Kullanıcı etkilendi mi? Hipotez çöktüyse açığı buldun demektir.
5. **Otomatize Et ve Düzelt:** Mimariyi güçlendir ve testin düzenli çalışmasını sağla.

RTO vs. RPO: Felaket Kurtarma Metrikleri

- **RTO (Recovery Time Objective):** Sistem ne kadar süre kapalı kalabilir? (Hedef: < 5 dakika).
- **RPO (Recovery Point Objective):** Şirket ne kadar veri kaybına dayanabilir? (Hedef: 0 işlem).

Kaçınılması Gereken Anti-Pattern'ler ve Tuzaklar

- **Gözleme Dayalı Olmayan Kaos:** İzleme panoları ve uyarıları tam olmayan bir sisteme hata basıp deneyi körü körüne bir felakete çevirmek.
- **İlk Günden Büyük Patlama:** Test ortamında hiç denemeden ilk günden ana canlı veritabanını kapatmaya kalkışmak.

- **Kağıt Üstünde Felaket Planı:** Beş yıldır hiç tatbikatı yapılmamış, dolapta duran 200 sayfalık tozlu 'Felaket Kurtarma Kitapçığı'.

> 🎯 **MİMARIN EYLEM PLANI: BU HAFTAKİ SOMUT ADIMLAR**
>
> - **1. Adım:** Test ortamında bir 'Kaos Günü' düzenle: Üçüncü parti ödeme servisinin bağlantısını kesip arayüzün nasıl davrandığını izle.
> - **2. Adım:** Test ortamında rastgele konteyner öldürmeyi otomatikleştir (Chaos Mesh ile).
> - **3. Adım:** Veritabanı devretme (failover) tatbikatı yap: Otomatik yedek sunucuya geçişin gerçekten çalıştığını doğrula.

Mimari Çıkarımlar ve Hile Tablosu

- Kaosun olup olmayacağını siz seçemezsiniz; kaos sizi seçer. Kaos mühendisliği ne zaman ve nasıl olacağını seçmenizi sağlar.
- Test edilmemiş bir yedek veya otomatik geçiş mekanizması sadece tatlı bir hayaldir.
- Kontrollü küçük deneylerle sisteme olan güveninizi adım adım inşa edin.

> 🎙️ **UDEMY & NOTEBOOKLM TARTIŞMA VE ÖDEV SORUSU**
>
> **Ders Değerlendirmesi ve Ödev:** Ekibiniz en son ne zaman ana veritabanı çöktüğünde ne olacağını test etti? Otomatik geçiş çalıştı mı yoksa manuel müdahale mi gerekti?